

Basic GLMM example

James Foster

2026-06-02

Contents

Basic GLMM example	1
Simulate data	1
Save and load data	3
Plot the binomial data	3
Fit GLMMs	4
Model comparison	5
Model reduction	5
Post-hoc comparisons	6
Prediction plots	7

```
library(lme4)
set.seed(17070523)
knitr::opts_chunk$set(echo = TRUE)
```

Basic GLMM example

This document simulates binomial data and fits a logistic mixed-effects model using `glmer()`.

Simulate data

We generate response data for individual animals, stimuli, and two stimulus types.

```
n_animals <- 10
n_levels <- 7
n_types <- 2
replicates <- 20

pop_mean <- 5.00
pop_sd <- 1.00
slope_mean <- 3.00
slope_sd <- 1.03
```

```

type_mean <- 2.50
type_slope <- 3.5
type_animal_sd <- 0.55
extra_error_sd <- 0.10

animal <- rep(x = 1:n_animals, times = n_levels)
animal <- sort(animal)
animal <- rep(x = animal, times = n_types)
stimulus <- rep(x = 1:n_levels - 1, times = n_animals)
stimulus <- rep(x = stimulus, times = n_types)
stype <- rep(x = 1:n_types, times = n_animals * n_levels)
stype <- sort(stype)

animal <- rep(x = animal, times = replicates)
stimulus <- rep(x = stimulus, times = replicates)
stype <- rep(x = stype, times = replicates)

base_bias_animal <- rnorm(n = n_animals, mean = pop_mean, sd = pop_sd)
slope_animal <- rnorm(n = n_animals, mean = slope_mean, sd = slope_sd)
type_animal <- rnorm(n = n_animals, mean = 1.0, sd = type_animal_sd)

response_y <- rep(x = NA, length = n_animals * n_levels * n_types * replicates)
for (ii in seq_along(response_y)) {
  response_y[ii] <- base_bias_animal[animal[ii]] +
    (stype[ii] - 1) * type_mean +
    (slope_animal[animal[ii]] +
     (stype[ii] - 1) * type_slope * type_animal[animal[ii]]) * stimulus[ii] +
    rnorm(n = 1, mean = 0, sd = extra_error_sd)
}

correct_incorrect <- rbinom(n = length(response_y),
                           size = 1,
                           prob = plogis(q = response_y - mean(response_y)))

sim_data <- data.frame(
  correct_incorrect = correct_incorrect,
  animal = LETTERS[animal],
  stimulus = stimulus,
  type = factor(stype, labels = c('alpha', 'beta'))
)

summary(sim_data)

```

```

## correct_incorrect  animal      stimulus  type
## Min.   :0.0000    Length:2800    Min.   :0   alpha:1400
## 1st Qu.:0.0000    Class :character  1st Qu.:1   beta :1400
## Median :0.0000    Mode  :character  Median :3
## Mean   :0.3886                    Mean   :3
## 3rd Qu.:1.0000                    3rd Qu.:5
## Max.   :1.0000                    Max.   :6

```

Save and load data

```
root_dir <- file.path(dirname(knitr::current_input(dir = TRUE)), 'Simulated Data')
file_name <- 'simulated_binomial_data.csv'
write.csv(file = file.path(root_dir, file_name), x = sim_data, row.names = FALSE)

dta <- read.csv(file = file.path(root_dir, file_name), header = TRUE)
dta <- within(dta, {
  animal <- factor(animal)
  type <- factor(type)
})

summary(dta)
```

```
## correct_incorrect    animal      stimulus    type
## Min.      :0.0000    A      : 280    Min.      :0    alpha:1400
## 1st Qu.:0.0000    B      : 280    1st Qu.:1    beta :1400
## Median :0.0000    C      : 280    Median :3
## Mean      :0.3886    D      : 280    Mean      :3
## 3rd Qu.:1.0000    E      : 280    3rd Qu.:5
## Max.      :1.0000    F      : 280    Max.      :6
##                      (Other):1120
```

Plot the binomial data

We aggregate the proportion correct for each animal and stimulus level.

```
clz <- sapply(length(unique(dta$animal)),
              colorRampPalette(c('red', 'blue', 'cyan4', 'magenta4', 'orange'))))
clz <- sample(clz)

agg <- aggregate(correct_incorrect ~ stimulus * type * animal, data = dta,
                 FUN = mean)
head(agg)
```

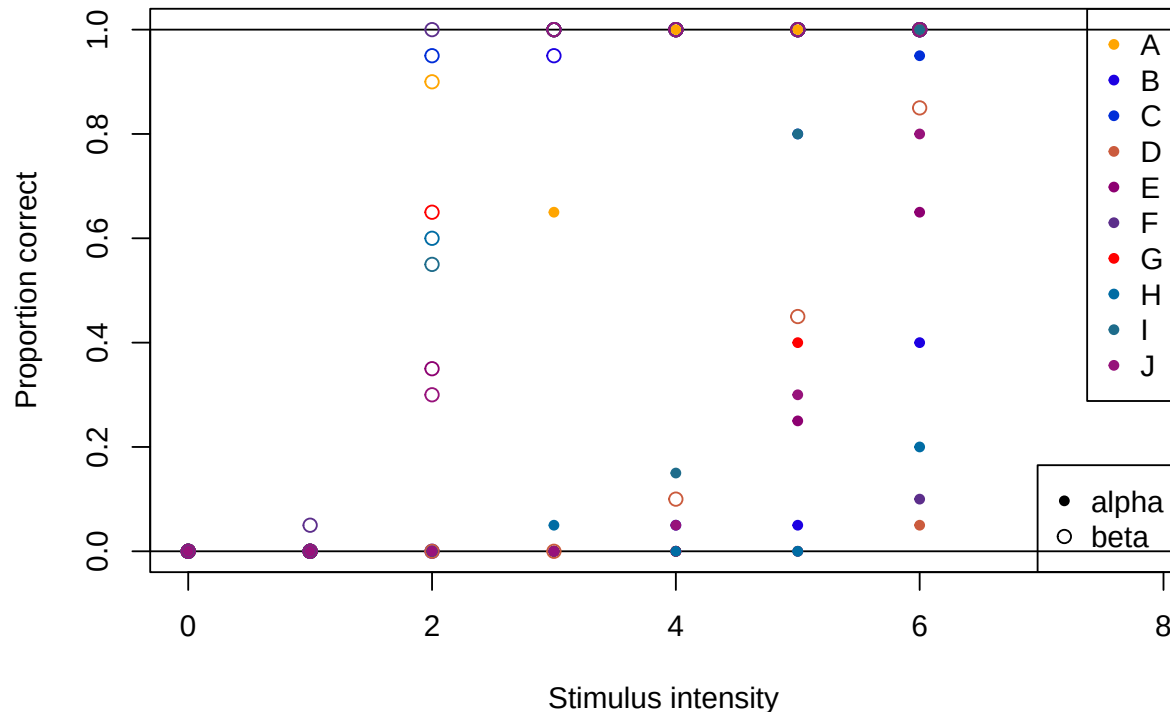
```
## stimulus type animal correct_incorrect
## 1      0 alpha      A              0.00
## 2      1 alpha      A              0.00
## 3      2 alpha      A              0.00
## 4      3 alpha      A              0.65
## 5      4 alpha      A              1.00
## 6      5 alpha      A              1.00
```

```
plot(NULL,
     xlab = 'Stimulus intensity',
     ylab = 'Proportion correct',
     xlim = range(stimulus) * c(0.7, 1.3),
     ylim = c(0, 1))
abline(h = c(0, 1))
for (aa in unique(agg$animal)) {
```

```

with(subset(agg, animal == aa), {
  points(
    x = stimulus,
    y = correct_incorrect,
    col = clz[which(LETTERS %in% aa)],
    pch = c(20, 21)[1 + type %in% levels(type)[2]]
  )
})
}
legend('topright', legend = unique(agg$animal), col = clz, pch = 20)
legend('bottomright', legend = unique(agg$type), pch = c(20, 21))

```



Fit GLMMs

We fit a maximal logistic mixed-effects model and a null random-effects model.

```

ctrl_opt <- glmerControl(optimizer = 'bobyqa')
glmm.max <- glmer(
  formula = correct_incorrect ~ stimulus * type + (1 + stimulus * type | animal),
  data = dta,
  control = ctrl_opt,
  family = binomial(link = 'logit')
)

```

```
## boundary (singular) fit: see help('isSingular')
```

```
glmm.null <- glmer(  
  formula = correct_incorrect ~ 1 + (1 | animal),  
  data = dta,  
  family = binomial(link = 'logit')  
)
```

Model comparison

```
extractAIC(glmm.max)[2]
```

```
## [1] 685.9584
```

```
extractAIC(glmm.null)[2]
```

```
## [1] 3587.245
```

```
anova(glmm.max, glmm.null, test = 'Chisq')
```

```
## Data: dta  
## Models:  
## glmm.null: correct_incorrect ~ 1 + (1 | animal)  
## glmm.max: correct_incorrect ~ stimulus * type + (1 + stimulus * type | animal)  
##          npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)  
## glmm.null    2 3587.2 3599.1 -1791.62    3583.2  
## glmm.max    14  686.0  769.1  -328.98     658.0 2925.3 12 < 2.2e-16 ***  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Model reduction

We compare models with and without the interaction, stimulus, and type terms.

```
no_int <- update(glmm.max, . ~ . - stimulus:type -  
  (1 + stimulus * type | animal) +  
  (1 + stimulus + type | animal))
```

```
## boundary (singular) fit: see help('isSingular')
```

```
anova(glmm.max, no_int)
```

```
## Data: dta  
## Models:  
## no_int: correct_incorrect ~ stimulus + type + (1 + stimulus + type | animal)  
## glmm.max: correct_incorrect ~ stimulus * type + (1 + stimulus * type | animal)  
##          npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)  
## no_int    9 719.80 773.24 -350.90    701.80  
## glmm.max  14 685.96 769.08 -328.98    657.96 43.844  5 2.491e-08 ***  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```

no_stimulus <- update(no_int, . ~ . - stimulus -
                      (1 + stimulus + type | animal) + (1 + type | animal))
anova(no_int, no_stimulus)

## Data: dta
## Models:
## no_stimulus: correct_incorrect ~ type + (1 + type | animal)
## no_int: correct_incorrect ~ stimulus + type + (1 + stimulus + type | animal)
##           npar      AIC      BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
## no_stimulus    5 2867.2 2896.89 -1428.6   2857.2
## no_int         9  719.8  773.24  -350.9    701.8 2155.4  4 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

no_type <- update(no_stimulus, . ~ . - type - (1 + type | animal) + (1 | animal))
anova(no_stimulus, no_type)

## Data: dta
## Models:
## no_type: correct_incorrect ~ (1 | animal)
## no_stimulus: correct_incorrect ~ type + (1 + type | animal)
##           npar      AIC      BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
## no_type       2 3587.2 3599.1 -1791.6   3583.2
## no_stimulus    5 2867.2 2896.9 -1428.6   2857.2 726.04  3 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Post-hoc comparisons

We use `emmeans` to compare overall type effects and slope differences.

```

library(emmeans)
library(pbkrtest)

emm_intercepts <- emmeans(glmm.max, specs = list(pairwise ~ type),
                          adjust = 'sidak')
summary(emm_intercepts)

## $'emmeans of type'
##   type emmean   SE df asymp.LCL asymp.UCL
## alpha  -6.0 0.943 Inf   -7.84    -4.15
## beta   10.4 4.170 Inf    2.18    18.52
##
## Results are given on the logit (not the response) scale.
## Confidence level used: 0.95
##
## $'pairwise differences of type'
##   1          estimate   SE df z.ratio p.value
## alpha - beta   -16.3 3.84 Inf  -4.254 <.0001
##
## Results are given on the log odds ratio (not the response) scale.

```

```
emm_slopes_interact <- emtrends(glmm.max, specs = list(pairwise ~ type),
                               var = 'stimulus', adjust = 'sidak')
summary(emm_slopes_interact)
```

```
## $'emmeans of type'
## type stimulus.trend SE df asymp.LCL asymp.UCL
## alpha          2.69 0.369 Inf      1.96      3.41
## beta           10.40 3.840 Inf      2.86     17.93
##
## Confidence level used: 0.95
##
## $'pairwise differences of type'
## 1 estimate SE df z.ratio p.value
## alpha - beta    -7.71 3.71 Inf  -2.077  0.0378
```

Prediction plots

We compute predicted values and plot the fitted response curves for each type.

```
if (Sys.info()[['sysname']] == 'Windows') {
  require(snow)
}
newdta <- with(dta,
               expand.grid(stimulus = seq(min(stimulus),
                                         max(stimulus),
                                         length.out = 20),
                           type = unique(type),
                           animal = unique(animal)))
prd <- predict(glmm.max, newdata = newdta, type = 'response')

pfun <- function(x) predict(x, newdata = newdta, type = 'response')
avail.cores <- parallel::detectCores() - 1
clt <- parallel::makeCluster(spec = avail.cores, type = 'PSOCK')
parallel::clusterExport(clt, list('glmm.max', 'dta', 'newdta'))

bt <- bootMer(
  glmm.max,
  FUN = pfun,
  nsim = 50,
  re.form = NULL,
  parallel = 'snow',
  ncpus = parallel::detectCores() - 1,
  cl = clt
)
parallel::stopCluster(clt)

pred_q <- aggregate(pred ~ stimulus * type,
                    data = with(bt, data.frame(newdta, pred = c(t(t)))),
                    FUN = quantile, probs = c(0.025, 0.975))
mod_mean <- aggregate(prd ~ stimulus * type,
                      data = cbind(newdta, prd),
                      FUN = function(x) plogis(mean(qlogis(x))))
```

```

param_data <- merge(merge(newdta, pred_q, all = TRUE), mod_mean)
mod_pred <- within(param_data, {
  mod_mean <- prd
  CI_02.5 <- pred[, '2.5%']
  CI_97.5 <- pred[, '97.5%']
  rm(list = c('pred', 'prd'))
})

agg <- aggregate(correct_incorrect ~ stimulus * type * animal,
  data = dta,
  FUN = mean)

plot(NULL,
  xlab = 'Stimulus intensity',
  ylab = 'Proportion correct',
  xlim = range(stimulus) * c(0.7, 1.3),
  ylim = c(0, 1))
abline(h = c(0, 1))
for (aa in unique(agg$animal)) {
  with(subset(agg, animal == aa), {
    points(x = stimulus, y = correct_incorrect,
      col = c('orange2', 'darkblue')[1 + type %in% 'alpha'], pch = 21)
  })
}
with(subset(mod_pred, type == 'alpha'), {
  polygon(x = c(sort(stimulus), sort(stimulus, decreasing = TRUE)),
    y = c(CI_02.5[order(stimulus)], CI_97.5[order(stimulus, decreasing = TRUE)]),
    col = adjustcolor('darkblue', alpha.f = 0.2),
    border = NA)
  lines(sort(stimulus),
    mod_mean[order(stimulus)],
    col = 'darkblue',
    lwd = 3)
})
with(subset(mod_pred, type == 'beta'), {
  polygon(x = c(sort(stimulus), sort(stimulus, decreasing = TRUE)),
    y = c(CI_02.5[order(stimulus)],
      CI_97.5[order(stimulus, decreasing = TRUE)]),
    col = adjustcolor('orange2', alpha.f = 0.2),
    border = NA)
  lines(sort(stimulus),
    mod_mean[order(stimulus)],
    col = 'orange2',
    lwd = 3)
})

```